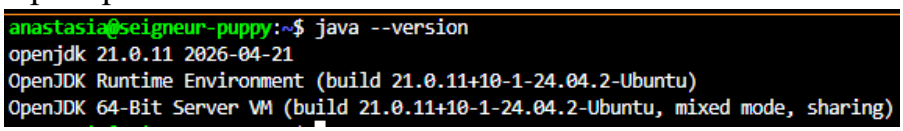
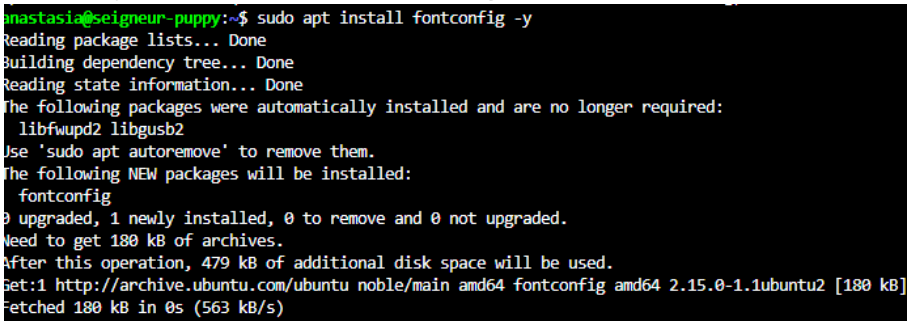
Установка и настройка Jenkins

1. Проверяю наличие Java с помощью команды java --version. После проверки обнаруживаю что ее нет, но высвечивается вариант установки. Устанавливаю 21 версию, так как она более стабильна – sudo apt install openjdk-21-jre-headless –y. Проверяю:



2. Также устанавливаю fontconfig



3. У меня не получилось установить Jenkins через ключи, так как останавливался из-за ошибки SSL, не мог связаться с сайтом, хотя пинг и прочее проходили. Помог способ через WAR файл. Ход установки был следующим:
   - Сначала устанавливаю Java скачивая openjdk-21-jdk
   - Далее создаю пользователя Jenkins: sudo useradd -d /var/lib/jenkins -s /bin/bash Jenkins
   - Создаю необходимые директории:
     - sudo mkdir -p /var/lib/jenkins
     - sudo mkdir -p /var/log/jenkins
     - sudo mkdir -p /opt/jenkins
   - Устанавливаем права:
     - sudo chown -R jenkins:jenkins /var/lib/jenkins
     - sudo chown -R jenkins:jenkins /var/log/jenkins
     - sudo chown -R jenkins:jenkins /opt/Jenkins
   - Скачиваю последнюю версию: sudo wget https://get.jenkins.io/war-stable/latest/jenkins.war -O /opt/jenkins/jenkins.war
   - Проверяем, что файл загрузился: ls -lh /opt/jenkins/jenkins.war
   - Создаем сервис systemd: sudo nano /etc/systemd/system/jenkins.service с следующим содержимым:

```
[Unit]
Description=Jenkins Continuous Integration Server
After=network.target

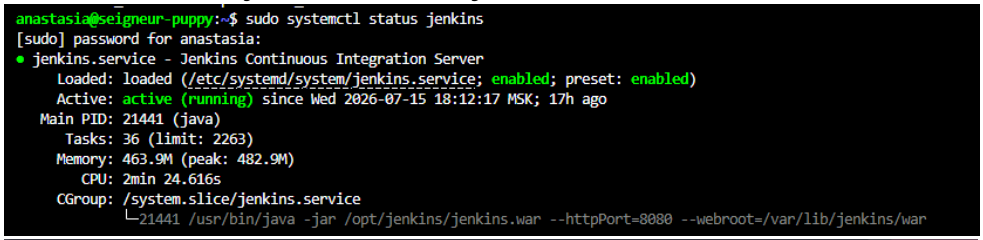
[Service]
User=jenkins
Group=jenkins
```

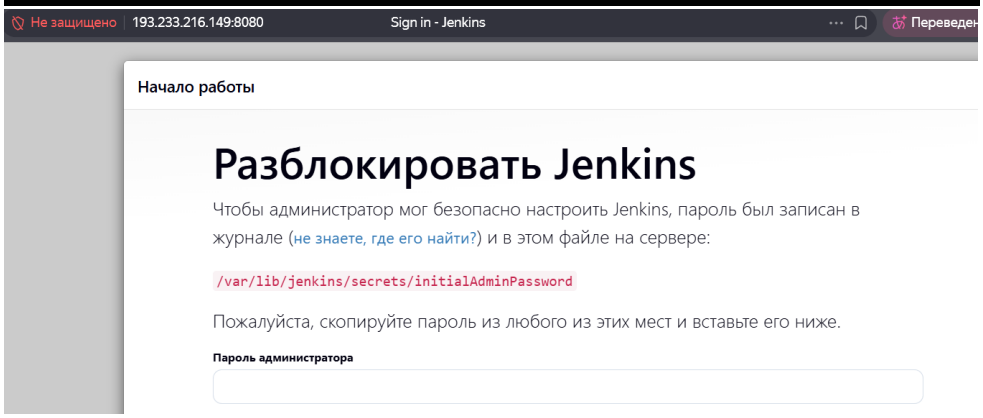
Type=simple
MainClass=jenkins.bootstrap.Bootstrap
Environment="JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64"
Environment="JENKINS_HOME=/var/lib/jenkins"
Environment="JENKINS_PORT=8080"
ExecStart=/usr/bin/java -jar /opt/jenkins/jenkins.war --httpPort=8080 --webroot=/var/lib/jenkins/war
WorkingDirectory=/opt/jenkins
Restart=on-failure
RestartSec=10

NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=strict
ProtectHome=true
ReadWritePaths=/var/lib/jenkins /var/log/jenkins

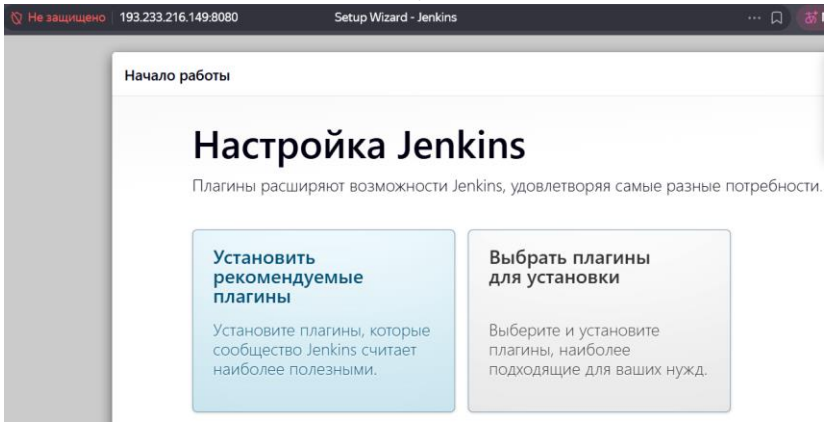
[Install]
WantedBy=multi-user.target

- Запускаю сервис:
  - sudo systemctl daemon-reload
  - sudo systemctl enable jenkins
  - sudo systemctl start jenkins
  - sudo systemctl status jenkins

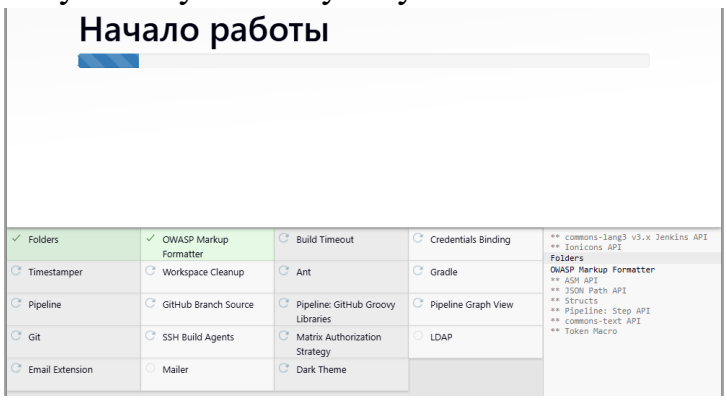




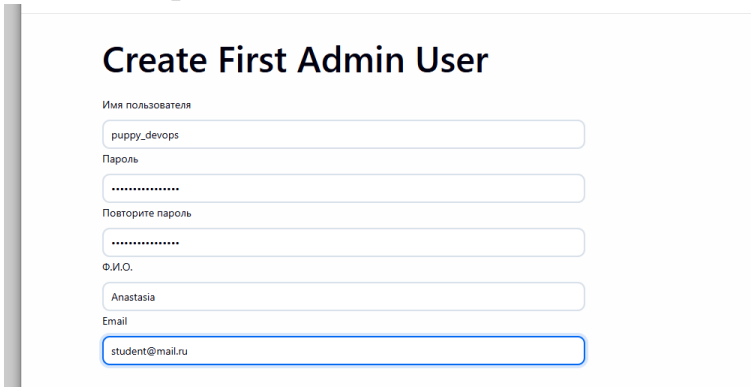
4. Чтобы разблокировать Jenkins необходимо катнуть пароль из файла initialAdminPassword по пути /var/lib/Jenkins/secrets



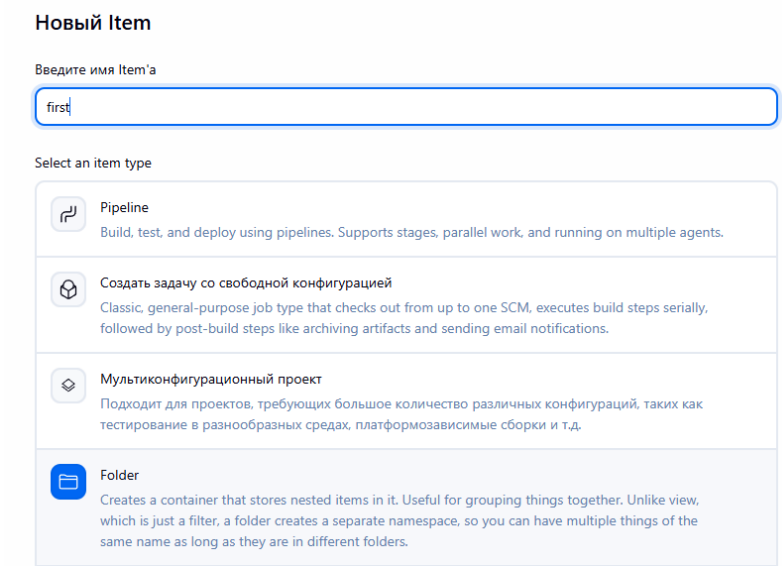
5. Запускаем установку по умолчанию:



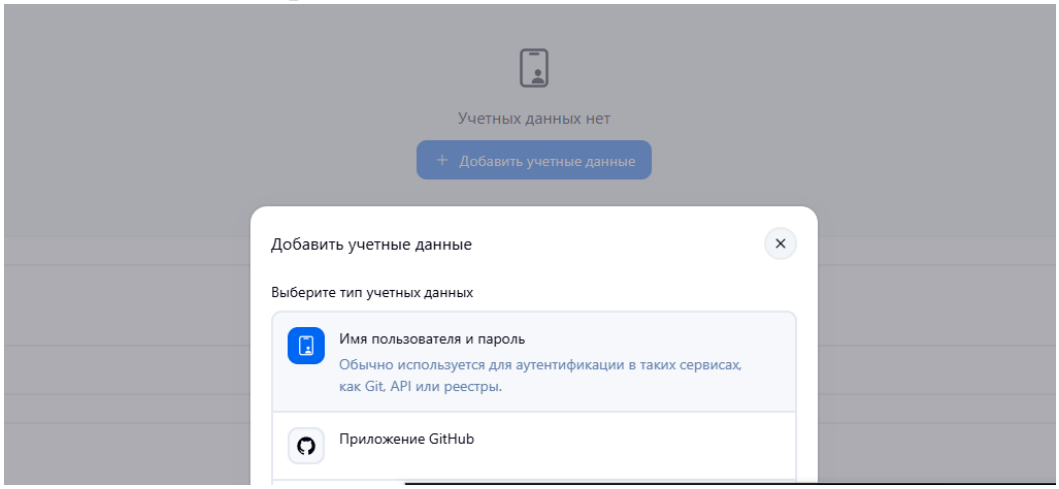
6. Создаю первого пользователя, админа:

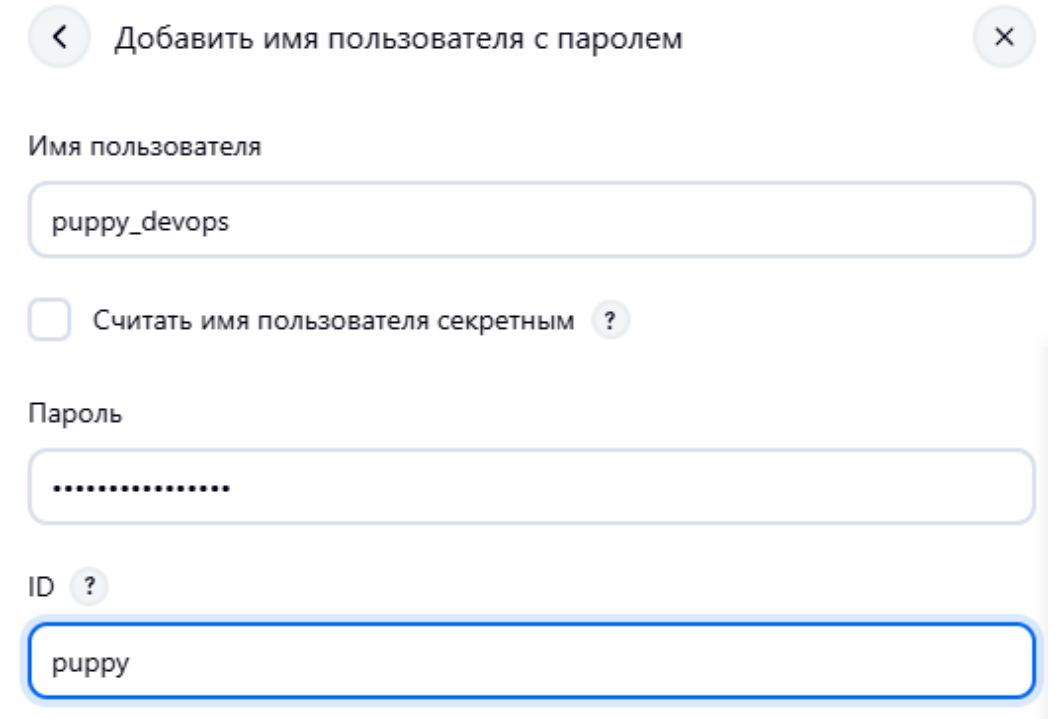


7. После авторизации необходимо создать первый folder, вообще folder помогает со структурированием джобов и созданием иерархии:



8. В категории Credentials задаю учетные данные, выбираю способ по имени пользователя и паролю:





Задание 4

1. Сначала создаю папку simple-java, в которой будет храниться приложение. Прогоняю git init, чтобы пользоваться Git и клонировать репозитории с Github.
2. На сервер клонировала с помощью git самое простое приложение, с которым можно ознакомиться по ссылке https://github.com/jenkins-docs/simple-java-maven-app.git. Также структура склонированной директории на скриншоте ниже

```
anastasia@seigneur-puppy:~/simple-java$ git clone https://github.com/jenkins-docs/simple-java-maven-app.git
Cloning into 'simple-java-maven-app'...
remote: Enumerating objects: 372, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (8/8), done.
remote: Total 372 (delta 7), reused 1 (delta 1), pack-reused 363 (from 4)
Receiving objects: 100% (372/372), 127.78 KiB | 1014.00 KiB/s, done.
Resolving deltas: 100% (130/130), done.
```

```
anastasia@seigneur-puppy:~/simple-java$ tree simple-java-maven-app/
simple-java-maven-app/
├── jenkins
│   ├── Jenkinsfile
│   └── scripts
│       └── deliver.sh
├── LICENSE.txt
├── pom.xml
├── README.md
└── src
    ├── main
    │   └── java
    │       └── com
    │           └── mycompany
    │               └── app
    │                   └── App.java
    └── test
        └── java
            └── com
                └── mycompany
                    └── app
                        └── AppTest.java
```

3. Для сборки приложения выбираю maven, чтобы им воспользоваться устанавливаю его себе sudo apt install maven –y:

```
anastasia@seigneur-puppy:~/simple-java$ mvn --version
Apache Maven 3.8.7
Maven home: /usr/share/maven
Java version: 21.0.11, vendor: Ubuntu, runtime: /usr/lib/jvm/java-21-openjdk-amd64
Default locale: en_US, platform encoding: UTF-8
OS name: "linux", version: "6.8.0-134-generic", arch: "amd64", family: "unix"
```

4. Как видно из скриншота со структурой директории клонированного проекта есть файл pom.xml. Этот файл главный для сборки, так как в нем хранится информация о свойствах, координатах проекта, зависимостей, плагинов приложения и прочее.
5. Перехожу в директорию проекта simple-java-maven-app и прописываю команду mvn compile. Но при сборке возникла ошибка версий, так как в файле настроек указана версия 3.9.9, а у меня 3.8.7, чтобы разрешить конфликт необходимо перейти в файл проекта pom.xml и найти правило, в котором не та версия и возникает конфликт. Как видно на скрине проблема

```
Caused by: org.apache.maven.plugin.MojoExecutionException:
Rule 0: org.apache.maven.enforcer.rules.version.RequireMavenVersion failed with message:
Detected Maven Version: 3.8.7 is not in the allowed range [3.9.9,).
```

```
<requireMavenVersion>
  <version>[3.9.9,)</version>
</requireMavenVersion>
```

6. Также необходимо установить компилятор javac, так как он идет отдельно от установочного пакета java в linux

```
anastasia@seigneur-puppy:~/simple-java/simple-java-maven-app$ javac -version
javac 21.0.11
```

7. После чего снова запускаю сборку mvn compile и она проходит успешно:

```
anastasia@seigneur-puppy:~/simple-java/simple-java-maven-app$ mvn compile
[INFO] Scanning for projects...
[INFO]
[INFO] ----------------------< com.mycompany.app:my-app >----------------------
[INFO] Building my-app 1.0-SNAPSHOT
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- maven-enforcer-plugin:3.6.3:enforce (enforce-maven) @ my-app ---
[INFO] Rule 0: org.apache.maven.enforcer.rules.version.RequireMavenVersion passed
[INFO] Rule 1: org.apache.maven.enforcer.rules.version.RequireJavaVersion passed
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ my-app ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory /home/anastasia/simple-java/simple-java-maven-app/src/main/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.15.0:compile (default-compile) @ my-app ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 1 source file with javac [debug release 21] to target/classes
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.538 s
[INFO] Finished at: 2026-07-18T21:49:49+03:00
[INFO] ------------------------------------------------------------------------
```

8. После сборки в папке проекта появляется новая директория target, в которой находятся все временные файлы и сгенерированные файлы, которые появляются в процессе работы над проектом, по сути промежуточная папка где собираются все действия, выполненные с приложением.

```
anastasia@seigneur-puppy:~/simple-java/simple-java-maven-app$ tree target/
target/
├── classes
│   └── com
│       └── mycompany
│           └── app
│               └── App.class
├── generated-sources
│   └── annotations
└── maven-status
    └── maven-compiler-plugin
        └── compile
            └── default-compile
                ├── createdFiles.lst
                └── inputFiles.lst
```

9. Теперь для проведения теста, необходимо установить зависимость Junit (фреймворк для проведения теста). Для этого необходимо перейти в pom.xml и первое поменять версию junit-jupiter-api с дефолтной, которая была при скачивании репозитория на 5.10.2 и добавить движок такой же версии для запуска тестов, который выделен на скриншоте

```
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-api</artifactId>
    <version>5.10.2</version>
    <scope>test</scope>
  </dependency>
  <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-engine</artifactId>
      <version>5.10.2</version>
      <scope>test</scope>
  </dependency>
</dependencies>
```

10. После установки зависимости, можно запустить команду mvn test, которая проведет тестирование и выведет результат, как можно увидеть по статусам, оно не упало, не выдало ошибку и не было пропущенных.

```
-------------------------------------------------------
 T E S T S
-------------------------------------------------------
Running com.mycompany.app.AppTest
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.005 sec

Results :

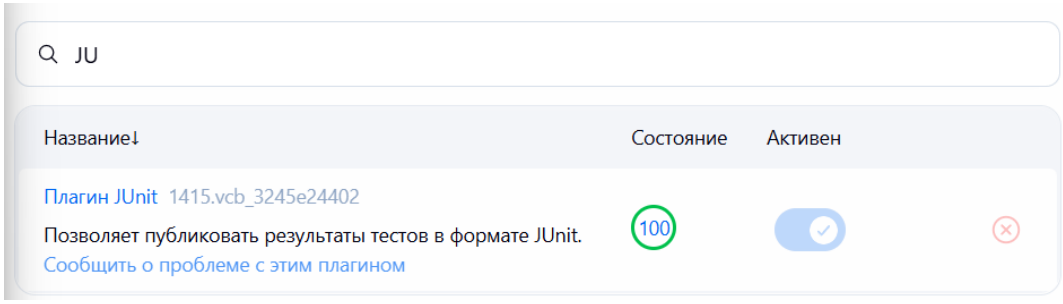
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0

[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  7.365 s
[INFO] Finished at: 2026-07-18T23:29:27+03:00
[INFO] ------------------------------------------------------------------------
```

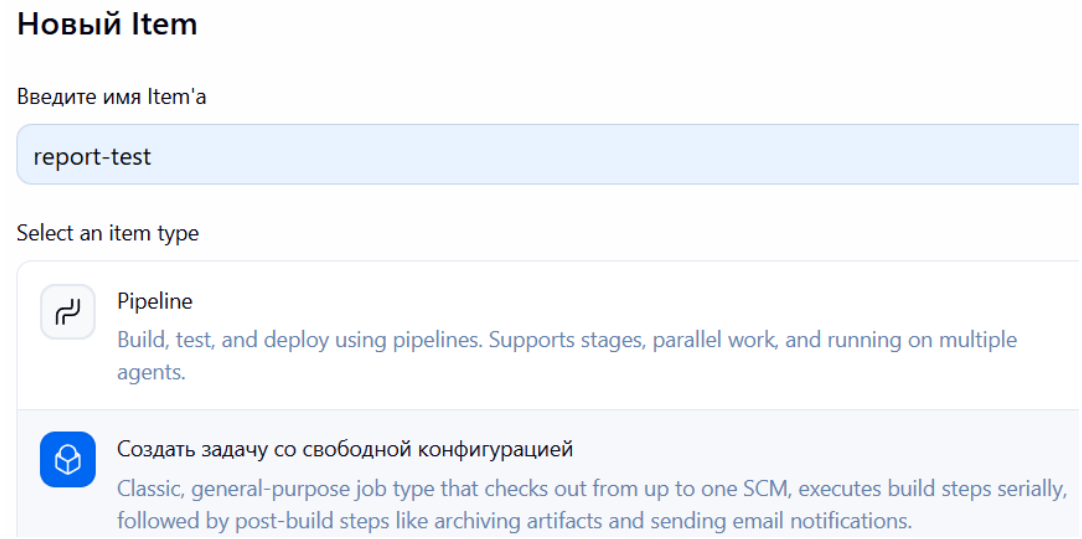
11. После запуска теста, в директории target появляется директория surefire-reports, в которой хранятся отчеты о проведенных тестах. Представлены в двух видах, первый это текстовый, а второй используется для публикации и тестирования в Jenkins (.xml)

```
anastasia@seigneur-puppy:~/simple-java/simple-java-maven-app$ tree target/surefire-reports/
target/surefire-reports/
├── com.mycompany.app.AppTest.txt
└── TEST-com.mycompany.app.AppTest.xml
```

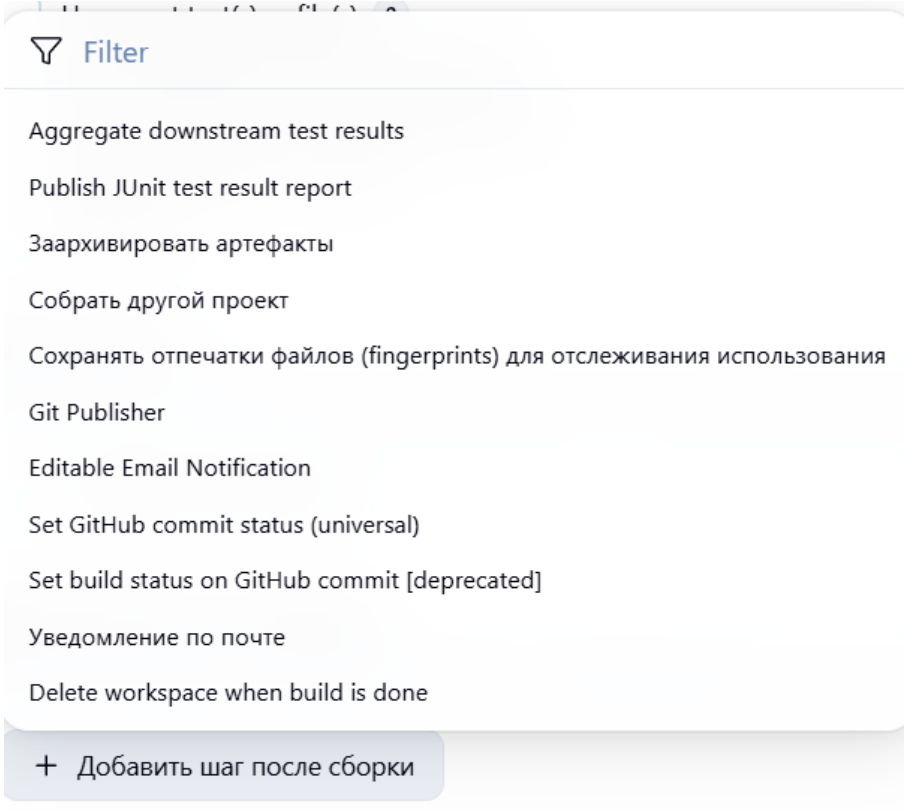
12. Теперь необходимо опубликовать результаты сборки и тестирования в веб-интерфейсе Jenkins. Для этого сначала нужно убедиться, что в веб версии Jenkins установлен плагин Junit, для этого переходим в настройки – plugins – installed plugins:



13. Также создаю новый item как на скриншоте:



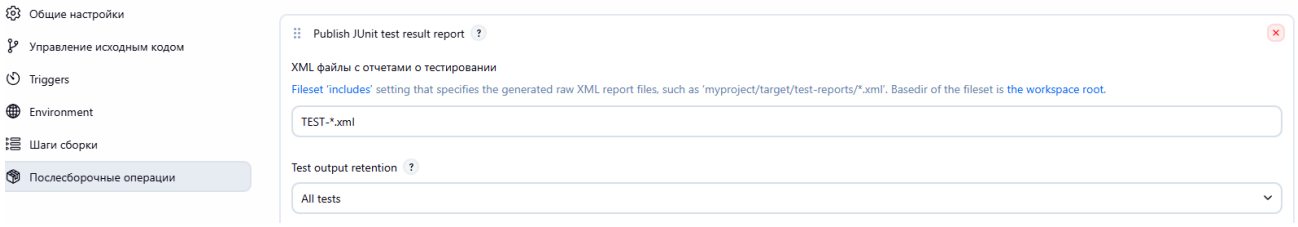
14. Далее перехожу в конфигурацию своего item. Выбираю пункт «Послесборные операции» и добавляю шаг под названием “Publish JUnit test result report”



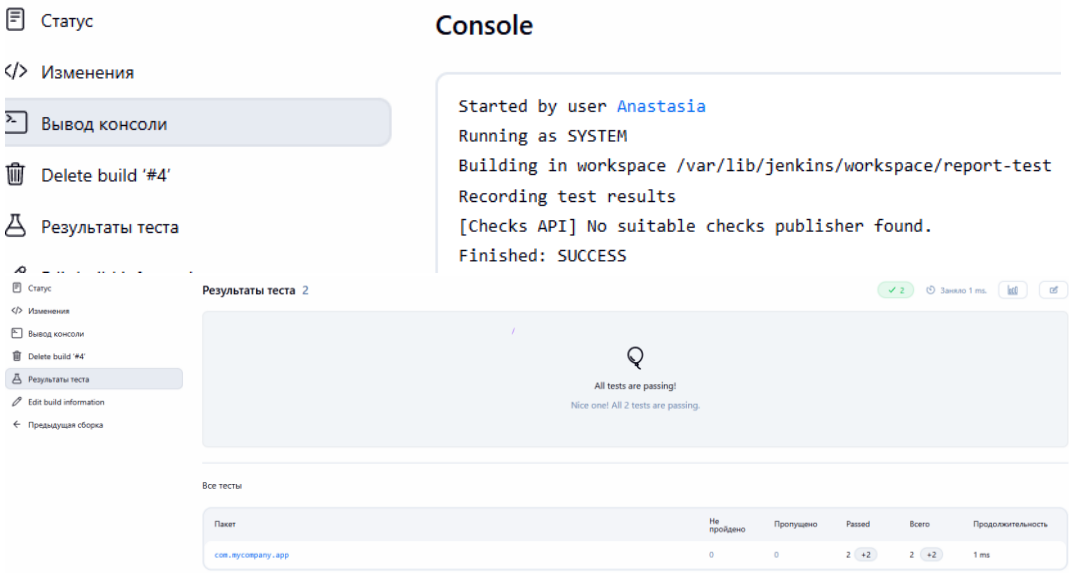
15. Для сборки отчетов и их передачу должен быть еще прописан плагин surefire в файле pom.xml, иначе сборка упадет в ошибку:

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.0.0</version>
</plugin>
```

16. Заполняю шаблон, ну как, вставляю только пункт с указанием пути до моего отчета сборки. Возникла проблема, что он не принимал путь, писал, что не существует, все из-за того, что при отработке джобы по отчетам с тестированием он пытается взять его по пути /var/lib/jenkins/workspace/report-test/, но у меня отчет лежит в ~/simple-java/simple-java-maven-app/target/surefire-reports. Поэтому для теста я перекинула файл с отчетом в папку выше:



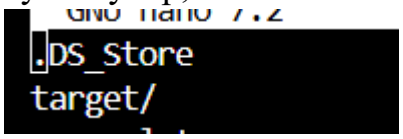
17. Запускаю сборку. Результат:



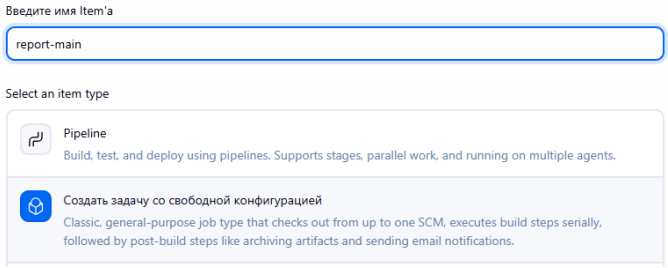
Я решила сначала пройти этапы настройки вручную, чтобы понять как это должно работать, по заданию необходимо автоматизировать, по сути в задании 5 я создам все с нуля с помощью джобы

Задание 5

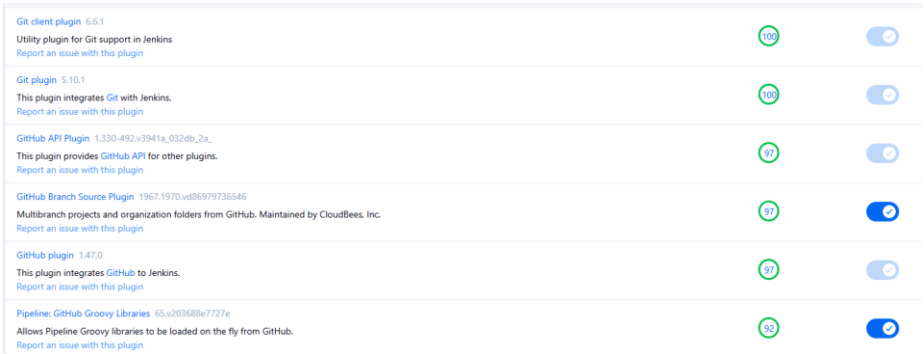
1. Переходим к автоматизации сборки и тестирования приложения, по сути те действия, что мы совершили сверху вручную должны выполняться при запуске джобы
2. Начну с того, что необходимо закину всю конфигурацию приложения в гит репозиторий кроме папки target, его вношу в .gitignore, так как в этой папке по сути мусор, постоянно изменяемые файлы



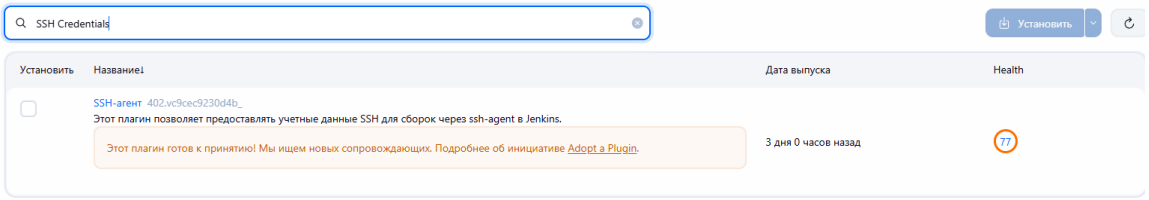
3. Создаю новый item, в которой будет происходить автоматическая сборка и тестирование



4. В командой строке создаю новую директорию Jenkins-main, в который будет выступать новым git репозиторием для работы над дженкинс, в гитхабе создаю репо Jenkins и ветку с таким же названием, куда буду пушить
5. Как только я все настроила, запушу туда приложение. Для тестирования сборки решила брать не исходный репозиторий приложения, а через свой репо делать, плюс как раз немного переделанный файл pom.xml
6. Итак перехожу в веб-интерфейс Jenkins в настройки – плагины – доступные плагины и проверяю наличие следующих плагинов: git, JUnit (он уже включен), Github



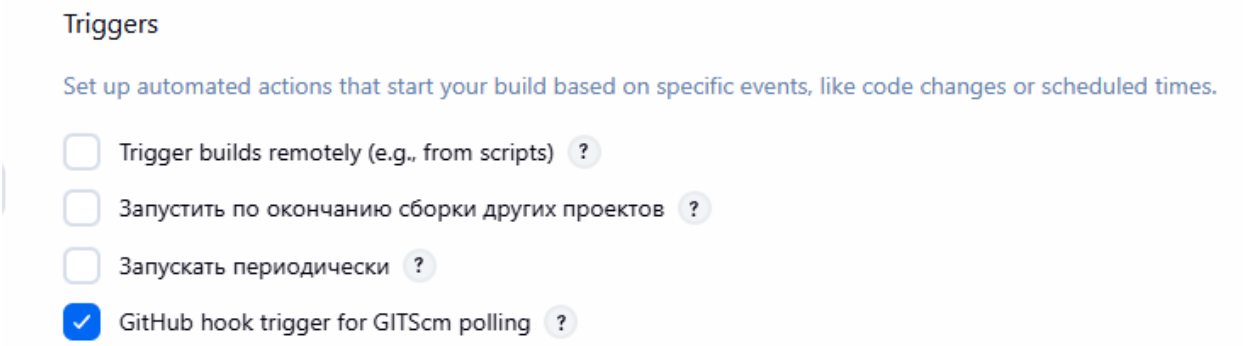
7. Перехожу в настройки конфигурации report-main во вкладку «управление исходным кодом», там выбираю Git, добавляю ссылку на мой репозиторий https://github.com/SeigneurPuppy1/jenkins . Далее в поле credentials добавляю ssh ключ перед этим перехожу в каталог плагинов и скачиваю SSH-agent и перезапускаю Jenkins:



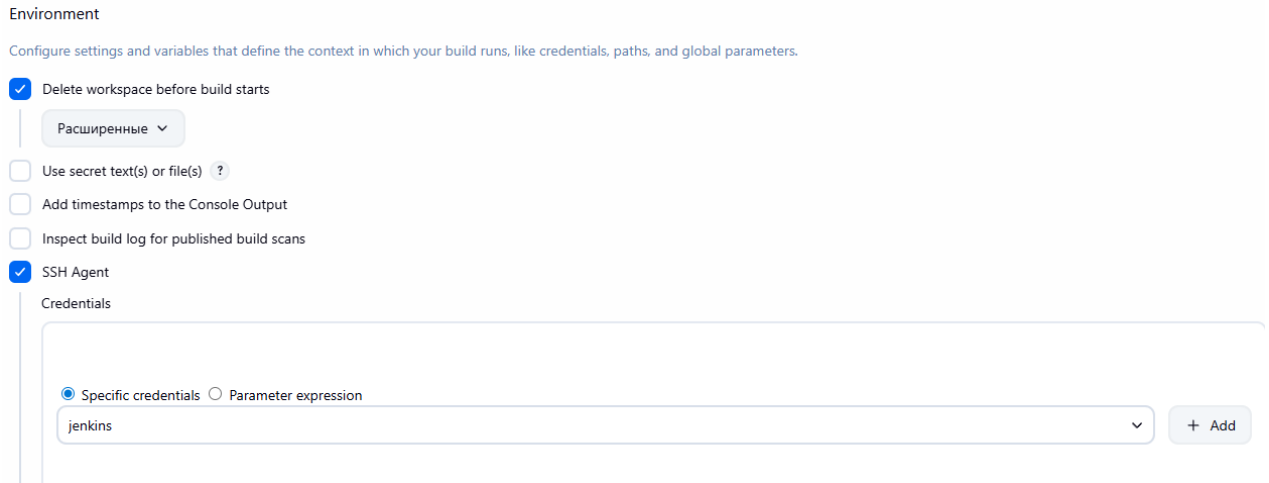
8. Создала SSH без заполнения полей, посмотрим, что будет



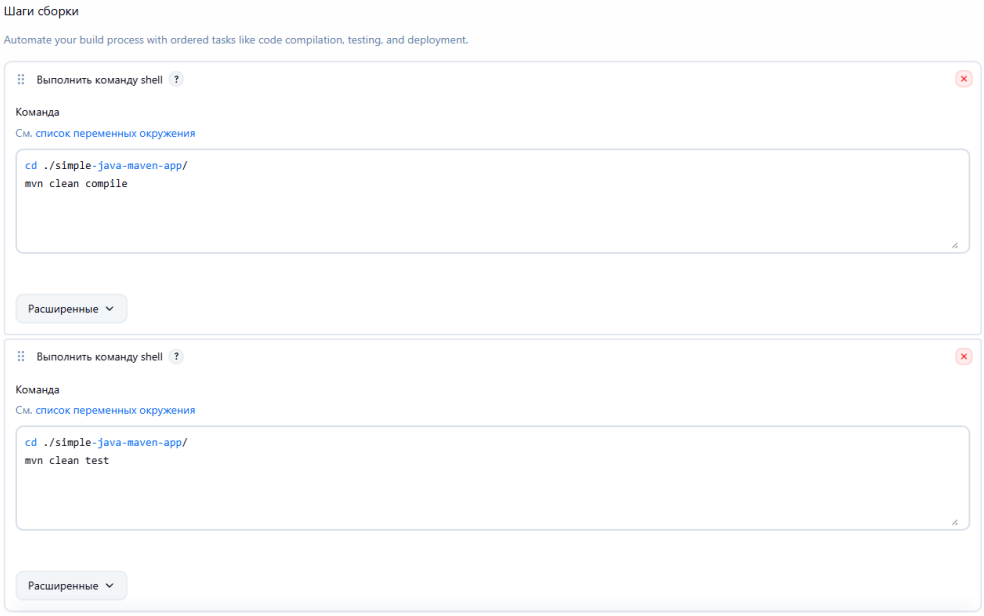
9. Настраиваю триггер, выбираю Github hook, удобно тем, что это не постоянный опрос репозитория, а однократный только когда в нем происходят изменения, если они есть он запускает сборку:



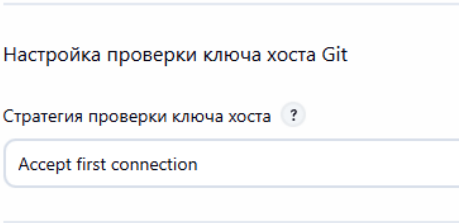
10. В этом пункте указала delete … чтобы при запуске джобы, когда он будет автоматически запускать сборку приложения, не было конфликтов с предыдущей версией, то есть начинает с чистого листа



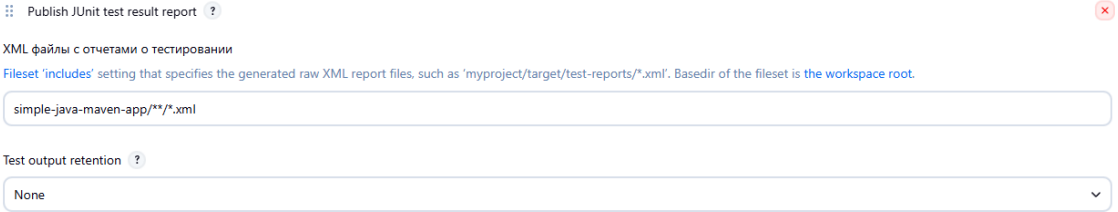
11. Прописываю две команды shell для сборки и тестирования приложения. Тут возник момент, что падал с ошибкой, так как файд pom.xml находился в подпапке simple-java-maven-app, поэтому в шаге сборке выбираем «выполнить команду shell» и прописываем запуск компиляции с переход в папку, где наш файл .xml. Второй шаг связан с тестом, там также явно указываем путь к файлу .xml, так как он не запоминает путь и по сути каждый шаг — это новый терминал



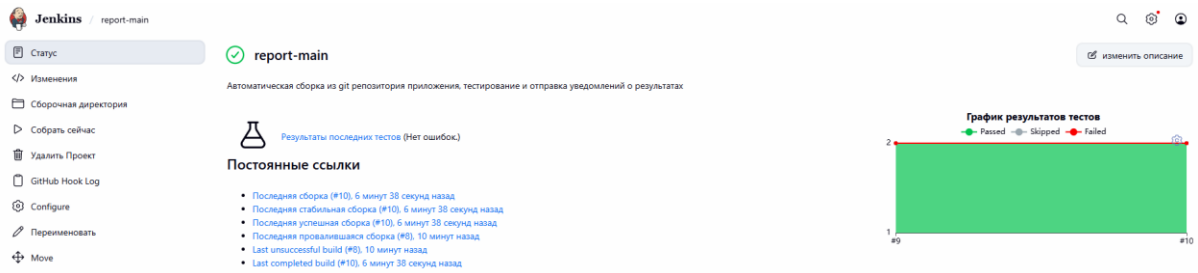
12. При пробной сборке выпала ошибка, первая связана с подключением ssh, так как по умолчанию в дженкинс во вкладке «Безопасность» установлена проверка ключей в файле known_hosts, его конечно можно настроить, но пока я попробую выбрать Accept first connection, автоматически добавит ключ в known_hosts, если ранее мы к нему не подключались:



13. Сбилась в конфигурации сборки поле про отчеты:



14. Сохраняю все изменения и запускаю сборку. Вот полный итог работы джобы



15. Результат вывода консоли:

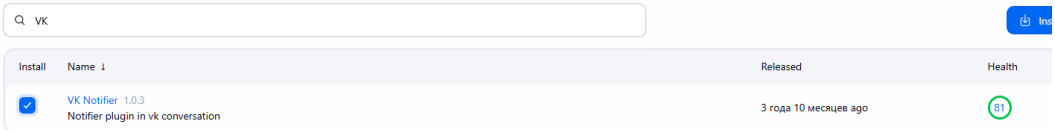
```
Запущено пользователем Anastasia
Запуск от имени SYSTEM
Сборка в рабочей области /var/lib/jenkins/workspace/report-main
[WS-CLEANUP] Удаление рабочей области проекта...
[WS-CLEANUP] Используется отложенное удаление...
[WS-CLEANUP] Готово
[ssh-agent] Поиск реализации ssh-agent...
$ ssh-agent
SSH_AUTH_SOCK=/tmp/ssh-4uBS3hpJbi37/agent.120688
SSH_AGENT_PID=120692
[ssh-agent] Запущено.
[ssh-agent] Использование учетных данных jenkins
Рекомендуемый инструмент для работы с Git: отсутствует
использование учетных данных ee47e155-f098-40bf-9139-5eda4d288ad6
Клонирование удаленного репозитория Git
Клонирование репозитория https://github.com/SeigneurPuppy1/jenkins.git
 > git init /var/lib/jenkins/workspace/report-main # timeout=10
Получение изменений из вышестоящего репозитория https://github.com/SeigneurPuppy1/jenkins.git
 > git --version # timeout=10
 > git --version # 'git version 2.43.0'
использование GIT_SSH для установки учетных данных
Проверка ключа хоста с помощью файла известных хостов автоматически принимает неизвестные ключи
 > git fetch --tags --force --progress -- https://github.com/SeigneurPuppy1/jenkins.git +refs/heads/*:refs/remotes/origin/* # timeout=10
 > git config remote.origin.url https://github.com/SeigneurPuppy1/jenkins.git # timeout=10
 > git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Избегайте повторной выборки
 > git rev-parse refs/remotes/origin/jenkins^{commit} # timeout=10
Проверка ревизии 4a89e7f2774d4d245c0aceaed264df6b1f4efe36 (refs/remotes/origin/jenkins)
 > git config core.sparsecheckout # timeout=10
 > git checkout -f 4a89e7f2774d4d245c0aceaed264df6b1f4efe36 # timeout=10
Сообщение о коммите: «Приложение для сборки и тестирования через Jenkins»
 > git rev-list --no-walk 4a89e7f2774d4d245c0aceaed264df6b1f4efe36 # timeout=10
[report-main] $ /bin/sh -xe /tmp/jenkins446371576102214439.sh
+ cd ./simple-java-maven-app/
+ mvn clean compile
[ [1;34mINFO [m] Сканирование проектов...
[ [1;34mINFO [m]
[ [1;34mINFO [m]  [1m----------------------<  [0;36mcom.mycompany.app:my-app [0;1m >---------------------- [m
[ [1;34mINFO [m]  [1mСборка my-app 1.0-SNAPSHOT [m
[ [1;34mINFO [m]  [1m--------------------------------[ jar ]--------------------------------- [m
[ [1;34mINFO [m]
[ [1;34mINFO [m]  [1m---  [0;32mmaven-clean-plugin:2.5:clean [m  [1m(default-clean) [m @  [36mmy-app [0;1m --- [m
[ [1;34mINFO [m]
[ [1;34mINFO [m]  [1m---  [0;32mmaven-enforcer-plugin:3.6.3:enforce [m  [1m(enforce-maven) [m @  [36mmy-app [0;1m --- [m
[ [1;34mINFO [m] Правило 0: org.apache.maven.enforcer.rules.version.RequireMavenVersion выполнено
[ [1;34mINFO [m] Правило 1: org.apache.maven.enforcer.rules.version.RequireJavaVersion выполнено
[ [1;34mINFO [m]
[ [1;34mINFO [m]  [1m---  [0;32mmaven-resources-plugin:2.6:resources [m  [1m(default-resources) [m @  [36mmy-app [0;1m --- [m
[ [1;34mINFO [m] Для копирования отфильтрованных ресурсов используется кодировка 'UTF-8'.
[ [1;34mINFO [m] пропустить несуществующий ресурсDirectory /var/lib/jenkins/workspace/report-main/simple-java-maven-app/src/main/resources
[ [1;34mINFO [m]
[ [1;34mINFO [m]  [1m---  [0;32mmaven-compiler-plugin:3.15.0:compile [m  [1m(default-compile) [m @  [36mmy-app [0;1m --- [m
[ [1;34mINFO [m] Перекомпиляция модуля из-за  [1mизменений в исходном коде [m.
[ [1;34mINFO [m] Компиляция 1 исходного файла с помощью javac [отладочная версия 21] в target/classes
[ [1;34mINFO [m]  [1m------------------------------------------------------------------------ [m
[ [1;34mINFO [m]  [1;32mBUILD SUCCESS [m
[ [1;34mINFO [m]  [1m------------------------------------------------------------------------ [m
[ [1;34mINFO [m] Общее время: 1,393 с
[ [1;34mINFO [m] Завершено: 2026-07-20T18:02:46+03:00
[ [1;34mINFO [m]  [1m------------------------------------------------------------------------ [m
[report-main] $ /bin/sh -xe /tmp/jenkins12730116294774687117.sh
+ cd ./simple-java-maven-app/
+ mvn clean test
[ [1;34mINFO [m] Сканирование проектов...
[ [1;34mINFO [m]
[ [1;34mINFO [m]  [1m----------------------<  [0;36mcom.mycompany.app:my-app [0;1m >---------------------- [m
[ [1;34mINFO [m]  [1mСборка my-app 1.0-SNAPSHOT [m
[ [1;34mINFO [m]  [1m--------------------------------[ jar ]--------------------------------- [m
[ [1;34mINFO [m]
[ [1;34mINFO [m]  [1m---  [0;32mmaven-clean-plugin:2.5:clean [m  [1m(default-clean) [m @  [36mmy-app [0;1m --- [m
[ [1;34mINFO [m] Удаление /var/lib/jenkins/workspace/report-main/simple-java-maven-app/target
[ [1;34mINFO [m]
[ [1;34mINFO [m]  [1m---  [0;32mmaven-enforcer-plugin:3.6.3:enforce [m  [1m(enforce-maven) [m @  [36mmy-app [0;1m --- [m
```

```
[ [1;34mINFO [m] T E S T S
[ [1;34mINFO [m] -------------------------------------------------------
[ [1;34mINFO [m] Запуск com.mycompany.app. [1mAppTest [m
[ [1;34mINFO [m]  [1;32mЗапущено тестов:  [0;1;32m2 [m, сбоев: 0, ошибок: 0, пропущено: 0, затраченное время: 0,076 с – в com.mycompany.app. [1mAppTest [m
[ [1;34mINFO [m]
[ [1;34mINFO [m] Результаты:
[ [1;34mINFO [m]
[ [1;34mINFO [m]  [1;32mЗапущено тестов: 2, сбоев: 0, ошибок: 0, пропущено: 0 [m
[ [1;34mINFO [m]
[ [1;34mINFO [m]  [1m------------------------------------------------------------------------ [m
[ [1;34mINFO [m]  [1;32mBUILD SUCCESS [m
[ [1;34mINFO [m]  [1m------------------------------------------------------------------------ [m
[ [1;34mINFO [m] Общее время: 2,830 с
[ [1;34mINFO [m] Завершено: 2026-07-20T18:02:50+03:00
[ [1;34mINFO [m]  [1m------------------------------------------------------------------------ [m
$ ssh-agent -k
unset SSH_AUTH_SOCK;
unset SSH_AGENT_PID;
echo Agent pid 120692 killed;
[ssh-agent] Stopped.
Recording test results
[Checks API] No suitable checks publisher found.
Finished: SUCCESS
```

16. Результаты теста:



17. Настрою уведомления через VK. Для этого проверяю наличие плагинов для уведомлений через VK, его не оказалось, поэтому устанавливаю:



18. Затем в вк создаем сообщество, далее по инструкции получаем API токен, далее необходимо узнать свой личный id, затем найти id сообщества, чтобы настроить отправку уведомлений
19. Далее переходим в нашу сборку и прописываем на стадии теста скрипт, чтобы он отправлял уведомление о сборке:

```
cd ./simple-java-maven-app/ || exit 1

set +e

mvn clean test

MVN_EXIT_CODE=$?

set -e

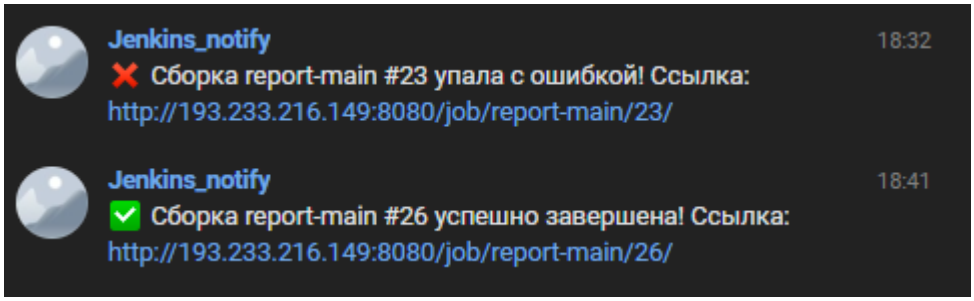
if [ $MVN_EXIT_CODE -eq 0 ]; then
    STATUS_ICON="✅"
    STATUS_TEXT="успешно завершена"
else
    STATUS_ICON="❌"
    STATUS_TEXT="упала с ошибкой"
fi

MESSAGE="${STATUS_ICON} Сборка ${JOB_NAME} #${BUILD_NUMBER} ${STATUS_TEXT}! Ссылка: ${BUILD_URL}"

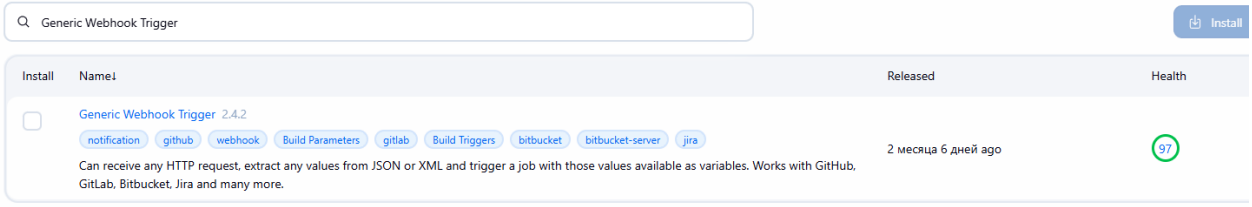
curl -s -X POST "https://api.vk.com/method/messages.send" \
  -d "peer_id=366737789" \
  -d "message=${MESSAGE}" \
  -d "random_id=$(date +%s)" \
  -d "access_token=vk1.a.3YVmEYJnfHle6vufOF8ULil_G1FAnwTM3A6iSLVQjb83Wvq2sR3IqPdCKImdrRCK2ojCcCNq_xC_oEGy6IqV2H52knUQee04IVJxvtwiODfUpendMM
  -d "v=5.199"

exit $MVN_EXIT_CODE
```

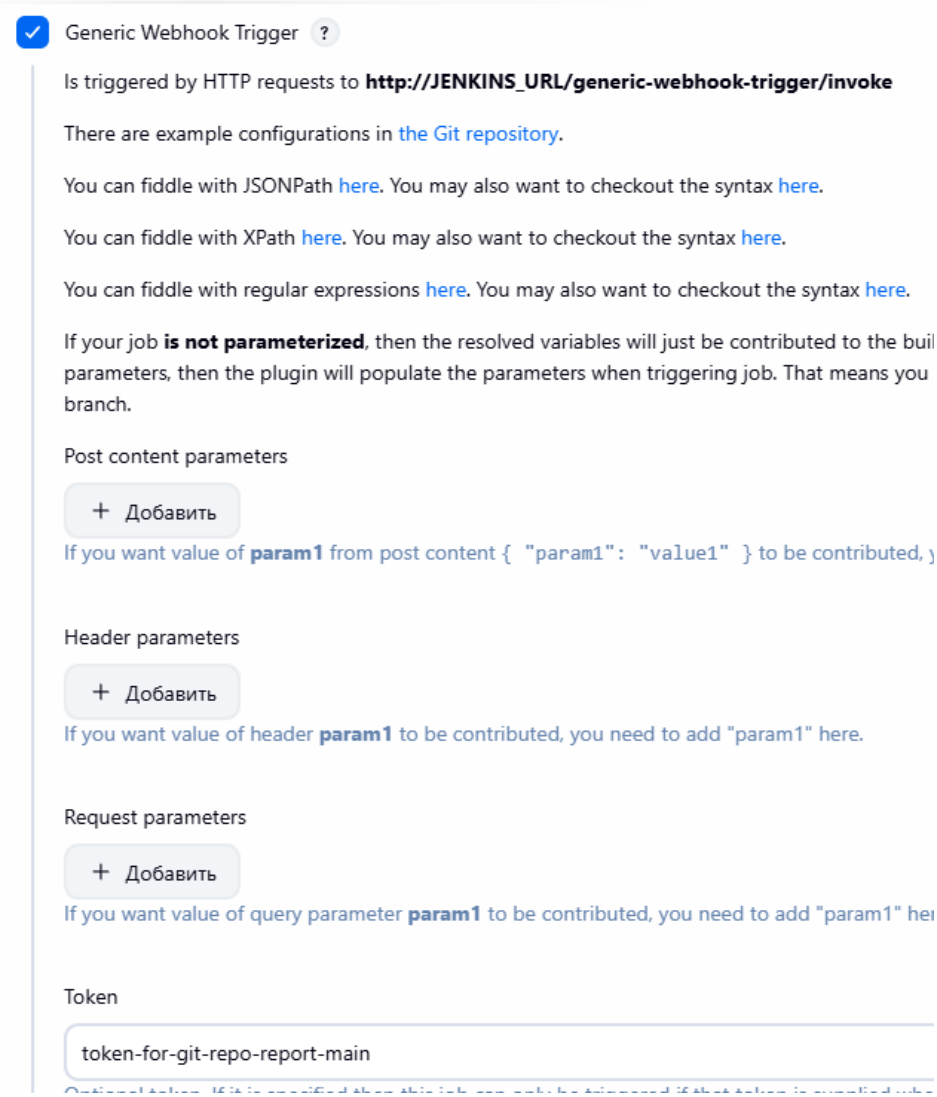
20. После того как нажимаем собрать сейчас, отрабатывает джоба и приходит вот такое сообщение в вк:



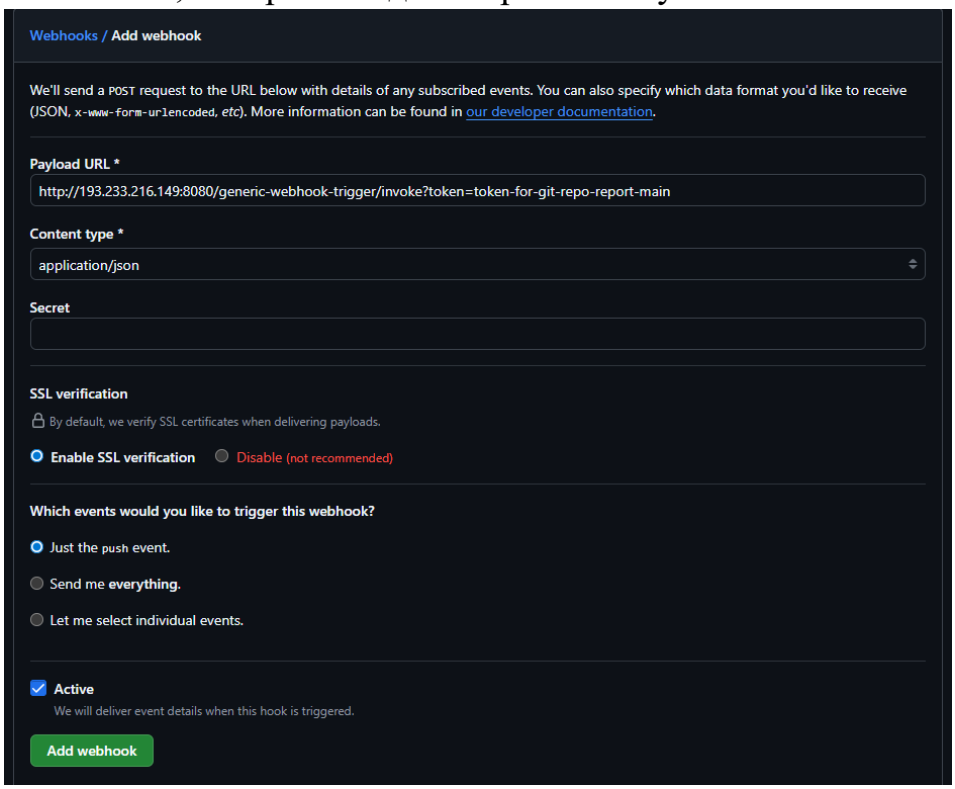
21. Теперь донастрою джобу, чтобы при изменениях в репозитории запускалась сборка, которую мы настроили выше. Для этого необходимо скачать плагин вебхуков, чтобы не дженкинс опрашивал репозиторий, а репозиторий бы сам оповещал об изменениях:



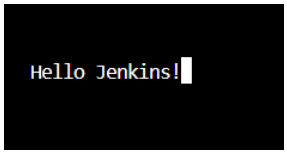
22. Перехожу в настройки джобы и в триггерах отмечаю Generic Webhook Trigger и придумываю свой токен. Остальное пока не трогаю:



23. Теперь необходимо настроить Webhook в Github. Захожу в настройки своего репозитория jenkins, нахожу пункт webhooks и нажимаю add webhook. Далее заполняю как на скриншоте. Адрес устанавливаю своего дженкинс и указываю свой токен, который создан в прошлом пункте



24. Теперь проверяем, зайду в свой репозиторий, в свой проект и в файл README.md добавлю строку:

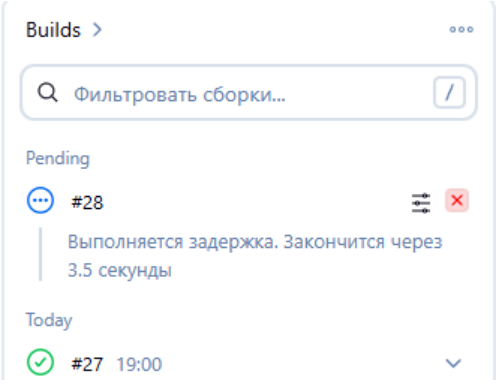


25. Теперь сохраняю, добавляю, коммичу и пушу на удаленный репозиторий, затем у нас должна запуститься сборка:

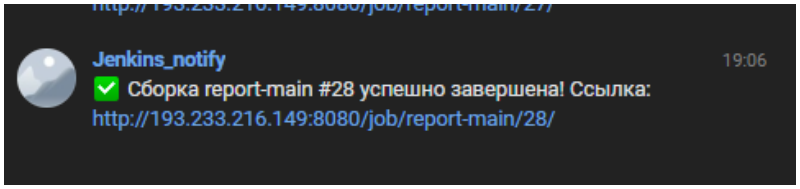
```
anastasia@seigneur-puppy:~/jenkins-main/simple-java-maven-app$ git status
On branch jenkins
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
anastasia@seigneur-puppy:~/jenkins-main/simple-java-maven-app$ git add .
anastasia@seigneur-puppy:~/jenkins-main/simple-java-maven-app$ git commit -m "Test webhook"
```

26. Вот запустилась сборка, после пуша:



27. Также вот оповещение:



28. Вот сам лог джобы:

```
Общая причина
Запуск от имени SYSTEM
Сборка в рабочей области /var/lib/jenkins/workspace/report-main
[WS-CLEANUP] Удаление рабочей области проекта...
[WS-CLEANUP] Используется отложенное удаление...
[WS-CLEANUP] Готово
[ssh-agent] Поиск реализации ssh-agent...
$ ssh-agent
SSH_AUTH_SOCK=/tmp/ssh-UTsd95hej0eI/agent.145372
SSH_AGENT_PID=145378
[ssh-agent] Запущено.
[ssh-agent] Использование учетных данных jenkins
Рекомендуемый инструмент для работы с Git: отсутствует
использование учетных данных ee47e155-f098-40bf-9139-5eda4d288ad6
Клонирование удаленного репозитория Git
Клонирование репозитория https://github.com/SeigneurPuppy1/jenkins.git
 > git init /var/lib/jenkins/workspace/report-main # timeout=10
Получение изменений из вышестоящего репозитория https://github.com/SeigneurPuppy1/jenkins.git
 > git --version # timeout=10
 > git --version # 'git version 2.43.0'
использование GIT_SSH для установки учетных данных
Проверка ключа хоста с использованием файла известных хостов автоматически принимает неизвестные ключи
 > git fetch --tags --force --progress -- https://github.com/SeigneurPuppy1/jenkins.git +refs/heads/*:refs/remotes/origin/* # timeout=10
 > git config remote.origin.url https://github.com/SeigneurPuppy1/jenkins.git # timeout=10
 > git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Избегайте повторной выборки
 > git rev-parse refs/remotes/origin/jenkins^{commit} # timeout=10
Проверка ревизии 349c68f52c73af9c23ad912b1999ca1ae31c3db6 (refs/remotes/origin/jenkins)
 > git config core.sparsecheckout # timeout=10
 > git checkout -f 349c68f52c73af9c23ad912b1999ca1ae31c3db6 # timeout=10
Сообщение о коммите: "Тестовый вебхук"
 > git rev-list --no-walk 4a89e7f2774d4d245c0aceaed264df6b1f4efe36 # timeout=10
[report-main] $ /bin/sh -xe /tmp/jenkins14374885737723906277.sh
+ cd ./simple-java-maven-app/
+ mvn clean compile
[ [1;34mINFO [m] Сканирование проектов...
[ [1;34mINFO [m]
[ [1;34mINFO [m]  [1m----------------------<  [0;36mcom.mycompany.app:my-app [0;1m >---------------------- [m
[ [1;34mINFO [m]  [1mСборка my-app 1.0-SNAPSHOT [m
[ [1;34mINFO [m]  [1m--------------------------------[ jar ]--------------------------------- [m
[ [1;34mINFO [m]
[ [1;34mINFO [m]  [1m---  [0;32mmaven-clean-plugin:2.5:clean [m  [1m(default-clean) [m @  [36mmy-app [0;1m --- [m
[ [1;34mINFO [m]
[ [1;34mINFO [m]  [1m---  [0;32mmaven-enforcer-plugin:3.6.3:enforce [m  [1m(enforce-maven) [m @  [36mmy-app [0;1m --- [m
[ [1;34mINFO [m] Правило 0: org.apache.maven.enforcer.rules.version.RequireMavenVersion выполнено
[ [1;34mINFO [m] Правило 1: org.apache.maven.enforcer.rules.version.RequireJavaVersion выполнено
[ [1;34mINFO [m]
[ [1;34mINFO [m]  [1m---  [0;32mmaven-resources-plugin:2.6:resources [m  [1m(default-resources) [m @  [36mmy-app [0;1m --- [m
[ [1;34mINFO [m] Использование кодировки 'UTF-8' для копирования отфильтрованных ресурсов.
[ [1;34mINFO [m] пропуск несуществующего каталога ресурсов /var/lib/jenkins/workspace/report-main/simple-java-maven-app/src/main/resources
[ [1;34mINFO [m]
[ [1;34mINFO [m]  [1m---  [0;32mmaven-compiler-plugin:3.15.0:compile [m  [1m(default-compile) [m @  [36mmy-app [0;1m --- [m
[ [1;34mINFO [m] Перекомпиляция модуля из-за  [1mизменений в исходном коде [m.
[ [1;34mINFO [m] Компиляция 1 исходного файла с помощью javac [отладочная версия 21] в target/classes
[ [1;34mINFO [m]  [1m------------------------------------------------------------------------ [m
[ [1;34mINFO [m]  [1;32mBUILD SUCCESS [m
[ [1;34mINFO [m]  [1m------------------------------------------------------------------------ [m
[ [1;34mINFO [m] Общее время: 1,548 с
[ [1;34mINFO [m] Завершено: 2026-07-21T19:06:48+03:00
[ [1;34mINFO [m]  [1m------------------------------------------------------------------------ [m
[report-main] $ /bin/sh -xe /tmp/jenkins18116595738686443268.sh
+ cd ./simple-java-maven-app/
+ set +e
+ mvn clean test
[ [1;34mINFO [m] Сканирование проектов...
[ [1;34mINFO [m]
[ [1;34mINFO [m]  [1m----------------------<  [0;36mcom.mycompany.app:my-app [0;1m >---------------------- [m
[ [1;34mINFO [m]  [1mСборка my-app 1.0-SNAPSHOT [m
[ [1;34mINFO [m]  [1m--------------------------------[ jar ]--------------------------------- [m
[ [1;34mINFO [m]
[ [1;34mINFO [m]  [1m---  [0;32mmaven-clean-plugin:2.5:clean [m  [1m(default-clean) [m @  [36mmy-app [0;1m --- [m
[ [1;34mINFO [m] Удаление /var/lib/jenkins/workspace/report-main/simple-java-maven-app/target
[ [1;34mINFO [m]
```

```
[ [1;34mINFO [m] T E S T S
[ [1;34mINFO [m] -------------------------------------------------------
[ [1;34mINFO [m] Запуск com.mycompany.app. [1mAppTest [m
[ [1;34mINFO [m]  [1;32m3апущено тестов:  [0;1;32m2 [m, сбоев: 0, ошибок: 0, пропущено: 0, затраченное время: 0,071 с — в com.mycompany.app. [1mAppTest [m
[ [1;34mINFO [m]
[ [1;34mINFO [m] Результаты:
[ [1;34mINFO [m]
[ [1;34mINFO [m]  [1;32m3апущено тестов: 2, сбоев: 0, ошибок: 0, пропущено: 0 [m
[ [1;34mINFO [m]
[ [1;34mINFO [m]  [1m------------------------------------------------------------------------ [m
[ [1;34mINFO [m]  [1;32mBUILD SUCCESS [m
[ [1;34mINFO [m]  [1m------------------------------------------------------------------------ [m
[ [1;34mINFO [m] Общее время: 2,915 с
[ [1;34mINFO [m] Завершено: 2026-07-21T19:06:52+03:00
[ [1;34mINFO [m]  [1m------------------------------------------------------------------------ [m
+ MVN_EXIT_CODE=0
+ set -e
+ [ 0 -eq 0 ]
+ STATUS_ICON=✅
+ STATUS_TEXT=успешно завершена
+ MESSAGE=✅ Сборка report-main #28 успешно завершена! Ссылка: http://193.233.216.149:8080/job/report-main/28/
+ date +%s
+ curl -s -X POST https://api.vk.com/method/messages.send -d peer_id=366737789 -d message=✅ Сборка report-main #28 успешно завершена! Ссылка: http://193.233.216.149:8080/job/report-main/28 /
-d random_id=1784650012 -d
access_token=vk1.a.3YVmEYJnfHle6vufOF8ULil_G1FAnwTM3A6iSLVQjbB3Wvq2sR3IqPdCKImdrRCK2ojCcCNq_xC_oE6y6IqV2H52knUQee04IVJxvtwiODfUpendMMa01rPAjqxnNSv1LxkVgtYzO10S2WjCQKgrq470CB3lLzas1ZgISzgS34FB
q42bwe6eieOXSaEj_97TtgLf9SBT9L84fuGFy_N_Bw -d v=5.199
{"response":15}+ exit 0
$ ssh-agent -k
unset SSH_AUTH_SOCK;
unset SSH_AGENT_PID;
echo Agent pid 145378 killed;
[ssh-agent] Stopped.
Запись результатов тестирования
[Checks API] Не найдено подходящего издателя проверок.
Завершено: УСПЕХ
```

Система	Плюсы (+)	Минусы (-)
Jenkins	• Бесплатный • 2000+ плагинов • Полный контроль • Любые интеграции • Работает offline	• Сложный в настройке • Устаревший UI • Требует админа • Конфликты плагинов • Сложно масштабировать
GitLab CI	• Всё в одном • Отличная визуализация • Встроенная безопасность • Auto DevOps • Хорошая масштабируемость	• Крутая кривая обучения • Привязка к GitLab • Ресурсоёмкий • Дорогой для больших команд
GitHub Actions	• Простая настройка • Огромный Marketplace • Бесплатно для public • Отличная интеграция с GitHub • Минимальный порог входа	• Слабая отладка • Сложно тестировать локально • Меньше визуализации • Может быть дорогим
CircleCI	• Очень быстрый • Продвинутый кэш Docker • SSH debugging • Разделение тестов • Удобный интерфейс	• Дорогой • Сложная система кредитов • Меньше кастомизации • Задержки сборок
Travis CI	• Простой • Бесплатно для open-source • Много языков • Хорошая документация	• Ограниченное время сборки • Дорогой для коммерции • Устаревает • Медленный
TeamCity	• Удобный интерфейс • Простая настройка • Мультиплатформенность • Регулярные обновления	• Дорогая лицензия • Требует ресурсов • Средняя поддержка
Bamboo	• Интеграция с Jira/Bitbucket • Disaster recovery • Простой деплой	• Мало интеграций • Дорогой • Нет cloud-версии • Замедляется при росте
Azure DevOps	• Комплексное решение • Интеграция с Azure • Масштабируемый • Управление backlog	• Крутая кривая обучения • Привязка к Microsoft • Медленные pipelines • Дорогой